LAPORAN UJIAN TENGAH SEMESTER

PEMROGRAMAN MOBILE

AniList Mini

[Tempatkan logo resmi fakultas/perguruan tinggi di sini]

Disusun oleh:

Nama: [NAMA MAHASISWA]

NIM: [NIM]

Program Studi/Jurusan: [PROGRAM STUDI/JURUSAN]

Perguruan Tinggi: [NAMA PERGURUAN TINGGI]

Tahun 2026

BAB I. PENDAHULUAN

1.1 Latar Belakang dan Gambaran Aplikasi

Perangkat mobile telah menjadi salah satu sarana untuk mengakses informasi secara praktis. Pengembangan aplikasi mobile dapat menggabungkan antarmuka interaktif dengan data yang diperoleh dari layanan jaringan. Informasi anime, misalnya, dapat disajikan dengan lebih mudah melalui aplikasi yang menampilkan katalog, poster, dan ringkasan karya.

Data dinamis dari REST API menghindarkan aplikasi dari ketergantungan pada daftar statis yang ditulis satu per satu. Dalam proyek UTS ini, AniList Mini dibuat menggunakan Flutter dan Dart dengan acuan struktur Jikan API v4. Karena public endpoint Jikan `api.jikan.moe` telah dihentikan pada 1 Oktober 2026, implementasi terbaru menggunakan Tenrai API v1 sebagai pengganti yang kompatibel dengan skema Jikan. Nama AniList Mini adalah nama aplikasi proyek; sumber data berisi katalog MyAnimeList, bukan layanan AniList.co.

Tujuan pengembangan adalah menerapkan permintaan HTTP GET, pemeriksaan status respons, parsing JSON ke objek Dart, asynchronous programming, dan penyajian data melalui antarmuka Flutter. Sasaran aplikasi ialah penggemar anime atau pengguna yang ingin menemukan referensi anime. Home menampilkan daftar populer dan pencarian; pengguna dapat membuka detail yang memuat poster, rating, jumlah episode, status, genre, tahun bila tersedia, serta sinopsis.

Implementasi saat ini memuat hingga 24 anime per permintaan dan belum menerapkan pagination. Pembatasan tersebut menjaga contoh tetap sederhana untuk pembelajaran.

1.2 Fitur Aplikasi

1. Top Anime. Home meminta data dari `/top/anime` dan menyajikannya dalam grid yang dapat dibangun secara efisien.

2. Pencarian judul. Pengguna mengetik kata kunci lalu menekan tombol Cari atau tombol Search pada keyboard. Request tidak dikirim pada setiap perubahan teks. Query dikirim menggunakan URL encoding.

3. Detail anime. Pemilihan kartu membuka route detail dan memanggil endpoint berdasarkan `mal_id`.

4. Refresh. Ikon refresh meminta daftar Top Anime kembali dan membersihkan kata kunci pencarian.

5. Status loading. `FutureBuilder` menyajikan indikator progres ketika permintaan belum selesai.

6. Penanganan error dan retry. Kesalahan jaringan, timeout, HTTP non-2xx dan parsing menghasilkan pesan ramah serta tombol untuk mengulang.

7. Empty state dan fallback poster. Daftar kosong diberi pesan “Anime tidak ditemukan”; gambar yang gagal dimuat diganti ikon.

Fitur favorit, filter genre, dan pagination belum dibuat dan tidak diklaim sebagai bagian implementasi.

BAB II. PEMBAHASAN

2.1 Persiapan Project dan Package

Project dibuat dengan Flutter 3.47.2 dan Dart 3.13.2. Perintah pengembangan dari awal:

```
flutter doctor
flutter create --platforms=android,web --project-name anilist_mini anilist_mini
cd anilist_mini
flutter pub add http
code .
flutter devices
flutter run
```

Package `http` digunakan untuk melakukan GET dan menerima objek respons. Package ini dipilih sebagai klien HTTP sederhana, tanpa menambah kerumitan arsitektur. `flutter_test` dan `flutter_lints` adalah dependency pengembangan dari template Flutter.

Struktur inti:

```
lib/
■■■ main.dart
■■■ models/anime_model.dart
■■■ services/anime_service.dart
■■■ pages/
    ■■■ home_page.dart
    ■■■ detail_page.dart
test/                       # unit/widget tests
android/                    # konfigurasi platform dan izin internet
web/                        # konfigurasi web Flutter
```

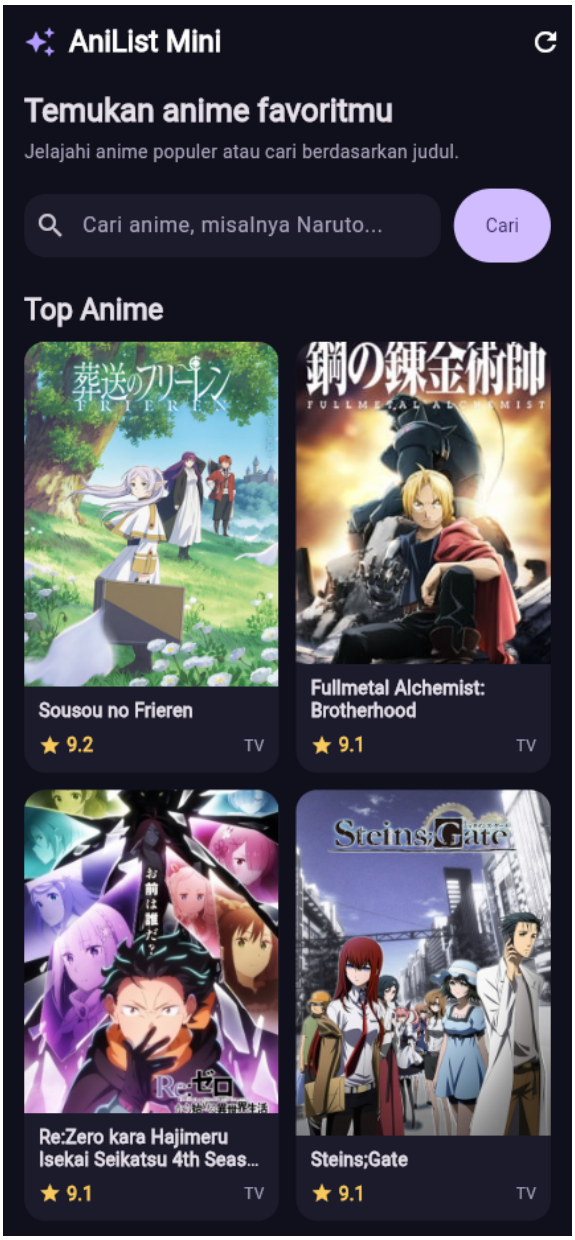
`main.dart` menginisialisasi tema dan halaman awal; model memetakan respons; service bertanggung jawab pada jaringan; halaman Home dan Detail menampilkan data. Folder `test/` berisi pengujian otomatis yang dipisahkan dari kode aplikasi.

2.2 Desain Antarmuka Pengguna (User Interface)

Antarmuka menerapkan mode gelap dengan warna latar #10101C, permukaan kartu #1C1B2B, ungu utama #8B5CF6, lavender #B39BFF, teks sekunder #A9A5BC, dan rating #F8C95C. Sudut membulat, jarak konsisten, ikon Material, serta poster dari URL API digunakan agar hierarki informasi mudah dibaca.

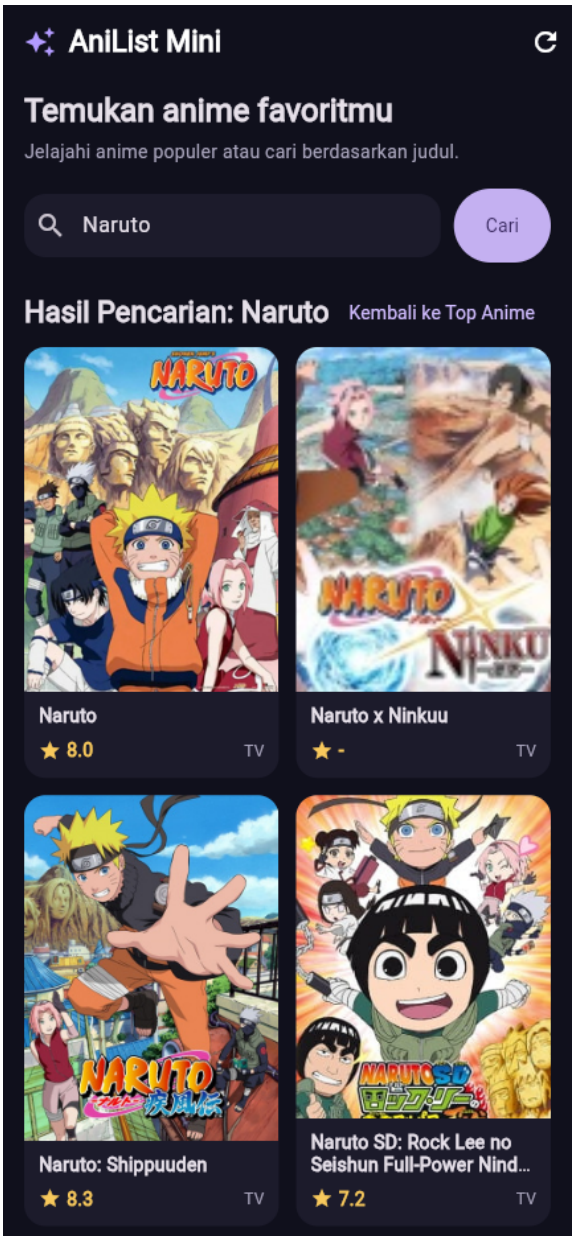
Home memuat judul aplikasi, sambutan, kolom pencarian, judul bagian dan `GridView.builder`. Grid menyesuaikan lebar layar: dua kolom pada layar sempit, bertambah pada layar lebar. Kartu memuat poster, judul, rating, dan tipe. Detail memakai konten yang dapat digulir dan menampilkan informasi lebih lengkap. Navigasi dibuka dengan `Navigator.push`; AppBar detail menyediakan kembali ke halaman sebelumnya.

Loading, error, empty state, serta kegagalan gambar memiliki tampilan alternatif. Aplikasi dirancang responsif, namun visual pada perangkat fisik perlu dikonfirmasi melalui pengujian manual.



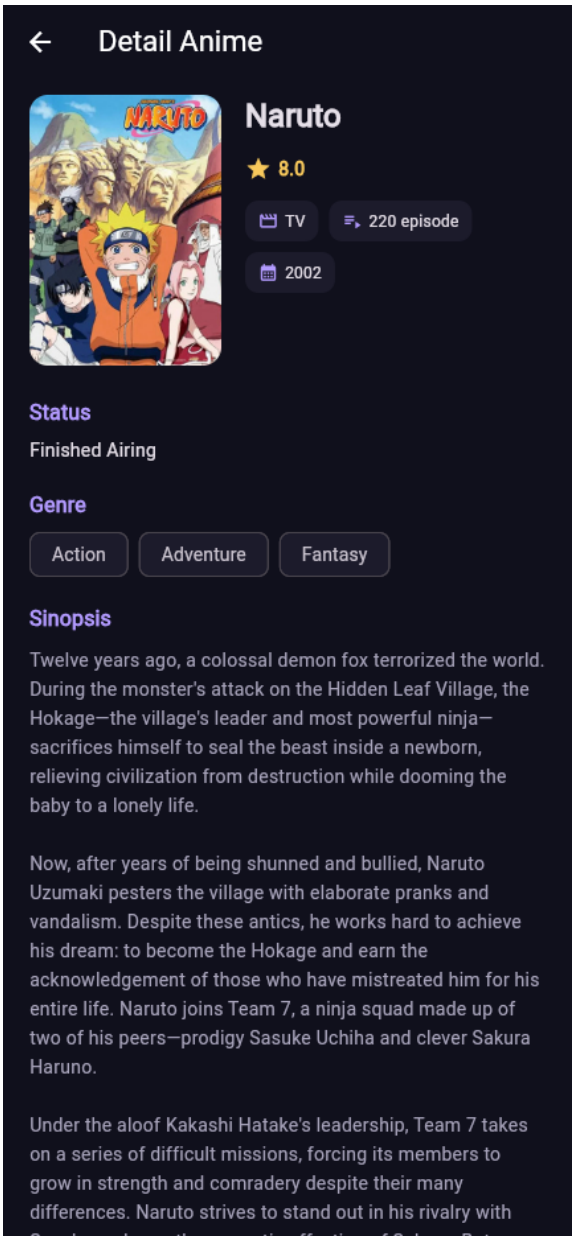
Gambar 2.1 Tampilan Halaman Utama AniList Mini.

Keterangan: Aplikasi dijalankan pada build web lokal dengan viewport mobile 430 × 932 piksel. Grid menampilkan poster, judul, rating, dan tipe anime dari endpoint live.



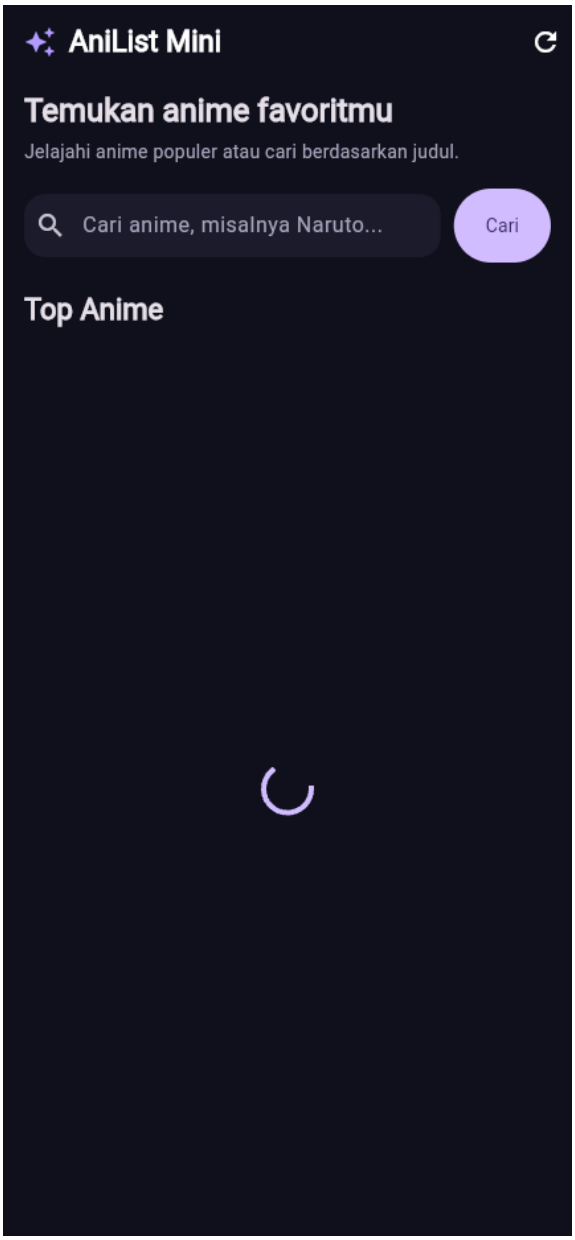
Gambar 2.2 Tampilan Hasil Pencarian Anime.

Keterangan: Hasil pencarian kata kunci “Naruto” menampilkan kartu Naruto, Naruto x Ninkuu, Naruto: Shippuuden, dan hasil terkait lainnya.



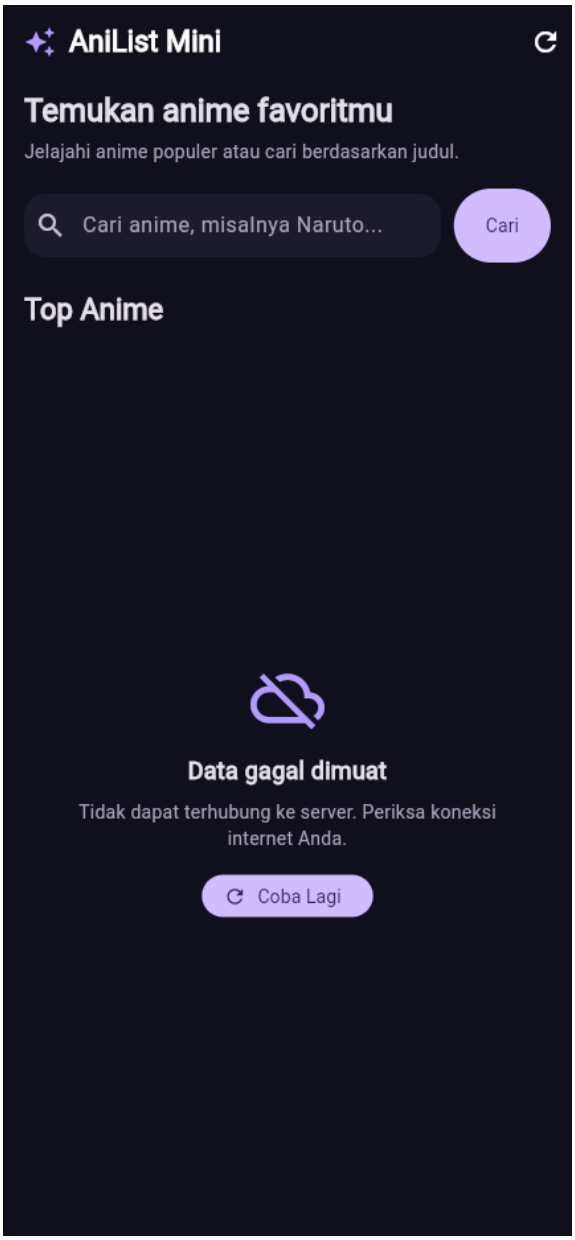
Gambar 2.3 Tampilan Detail Anime.

Keterangan: Halaman detail Naruto menampilkan poster, rating 8.0, tipe TV, 220 episode, tahun 2002, status Finished Airing, genre, dan sinopsis.



Gambar 2.4 Tampilan Loading Data.

Keterangan: CircularProgressIndicator ditangkap ketika respons endpoint sengaja ditunda dalam pengujian browser. Ini adalah state aplikasi aktual, bukan gambar rancangan.



Gambar 2.5 Tampilan Penanganan Error.

Keterangan: Request ke endpoint sengaja dibatalkan dalam pengujian browser. Aplikasi menampilkan pesan koneksi dan tombol Coba Lagi.

2.3 Implementasi Pengambilan dan Pengolahan Data dari API

A. Jikan API dan data

REST API menyediakan sumber daya melalui endpoint HTTP. Pada rancangan awal, Jikan API v4 ditetapkan sebagai sumber data. Dalam implementasi final, public endpoint Jikan `api.jikan.moe` sudah dihentikan pada 1 Oktober 2026. Oleh sebab itu, aplikasi memakai Tenrai API v1 yang menyatakan kompatibel dengan skema Jikan v4. Perubahan ini mempertahankan struktur data dan tujuan tugas, sementara aplikasi tetap dapat mengambil data live. Endpoint yang digunakan:

• `https://api.tenrai.org/v1/top/anime?limit=24` untuk daftar.

• `https://api.tenrai.org/v1/anime?q={query}&limit=24` untuk pencarian.

• `https://api.tenrai.org/v1/anime/{mal_id}` untuk detail.

Tugas UTS menetapkan Jikan API v4 sebagai acuan. Endpoint publik Jikan `api.jikan.moe` telah dihentikan pada 1 Oktober 2026. Oleh karena itu implementasi memakai Tenrai API v1 sebagai pengganti kompatibel skema Jikan v4, sehingga struktur model dan endpoint tetap relevan terhadap ketentuan tugas.

Objek anime memuat `mal_id`, `title`, `images.jpg.image_url`, `score`, `episodes`, `synopsis`, `status`, `genres`, `type`, serta `year`. Atribut yang dapat tidak tersedia diperlakukan null-safe.

B. HTTP request dan status

Service menggunakan package `http` untuk mengirim GET. Respons diperiksa dahulu: hanya kode 2xx diproses sebagai sukses; 429 diberi pesan rate-limit yang spesifik, sedangkan kode non-2xx lain menghasilkan pesan status HTTP. Request memiliki timeout 20 detik dan gangguan koneksi menghasilkan pesan yang dapat ditampilkan. Permintaan diberi jarak minimum 400 ms sebagai mitigasi sederhana, bukan jaminan melewati seluruh pembatasan API.

C. JSON parsing dan model

Respons diterima sebagai teks JSON. `jsonDecode` mengubahnya menjadi struktur Map/List Dart. Setiap Map kemudian diproses oleh `AnimeModel.fromJson`. Contoh kode yang merujuk file model proyek:

```
factory AnimeModel.fromJson(Map<String, dynamic> json) {
  final images = json['images'] as Map<String, dynamic>?;
  final jpg = images?['jpg'] as Map<String, dynamic>?;
  return AnimeModel(
    id: (json['mal_id'] as num?)?.toInt() ?? 0,
    title: json['title'] as String? ?? 'Judul tidak tersedia',
    imageUrl: jpg?['image_url'] as String? ?? '',
    score: (json['score'] as num?)?.toDouble() ?? 0,
    episodes: (json['episodes'] as num?)?.toInt(),
    // atribut lain: synopsis, status, genres, type, year
  );
}
```

Factory membaca objek bersarang menggunakan Map bertingkat. Operator ? menghindari akses properti pada null, ?? menyediakan nilai pengganti, dan nilai numerik JSON yang dapat berupa int atau double dikonversi melalui `num`. Untuk UI, score 0 diperlakukan sebagai belum tersedia; episode dan tahun tetap nullable.

D. Asynchronous programming dan FutureBuilder

HTTP memerlukan waktu. `Future<T>` mewakili nilai yang akan datang; `async` menandai fungsi asinkron dan `await` menunggu operasi tanpa membekukan antarmuka. Future disimpan pada state, bukan dibuat berulang di dalam `build`. `FutureBuilder` membaca Future yang sama dan memperbarui tampilan sesuai statusnya.

Contoh pola di halaman Home:

```
if (snapshot.connectionState == ConnectionState.waiting) {
  return const Center(child: CircularProgressIndicator());
}
if (snapshot.hasError) {
  return _MessageState(
    title: 'Data gagal dimuat',
    message: '${snapshot.error}',
    buttonText: 'Coba Lagi',
    onPressed: _retry,
  );
}
final animeList = snapshot.data ?? [];
if (animeList.isEmpty) {
  return _MessageState(
    title: 'Anime tidak ditemukan',
    message: 'Coba gunakan judul atau kata kunci lain.',
    buttonText: 'Tampilkan Top Anime',
    onPressed: _showTopAnime,
```

```
    );
  }
```

Kutipan diringkas dari implementasi aktual. Loading, error dan list kosong merupakan kondisi berbeda; list kosong adalah respons sukses tanpa item.

E. Penyajian data dan navigasi

`GridView.builder` membangun kartu berdasarkan indeks dan jumlah data. `Image.network` menggunakan URL poster API dengan `errorBuilder` sebagai fallback. `Navigator.push` membuka detail. Detail melakukan request tersendiri untuk `/anime/{id}` sehingga contoh mencakup pengambilan daftar dan pengambilan objek detail.

F. Pengujian aplikasi

Pengujian otomatis dilakukan dengan fixture dan `MockClient`, yang memeriksa parsing model, endpoint Tenrai, query encoding, detail, HTTP 429/500, respons JSON invalid, kesalahan jaringan, serta tampilan top anime dan hasil kosong. Selain itu, pemeriksaan live memakai service Flutter membuktikan tiga request: Top Anime menghasilkan 24 item dengan item pertama Sousou no Frieren, pencarian Naruto menghasilkan 24 item dengan hasil pertama Naruto, dan detail ID 5114 menghasilkan Fullmetal Alchemist: Brotherhood. Pemeriksaan live belum menggantikan pengujian visual manual pada perangkat.

No.	Fitur	Skenario	Hasil yang diharapkan	Hasil aktual	Status
1	Build aplikasi	`flutter build web`	Build selesai	Build web berhasil	Lulus build; bukan uji manual
2	Top Anime UI	Fixture dan endpoint live	Judul ditampilkan	Widget test lulus; Tenrai live mengembalikan 24 item dan item pertama Sousou no Frieren	Lulus otomatis dan live
3	Poster live	Buka build web di Chrome	Poster API muncul	Poster terlihat di Home, hasil pencarian dan detail	Lulus runtime web
4	Rating dan judul	Fixture model	Judul/rating tampil	Widget test menemukan judul dan 9.1	Lulus otomatis
5	Pencarian	Fixture, query encoding, dan request live Naruto	Hasil pencarian ditampilkan	Service test lulus; Tenrai live mengembalikan 24 hasil dan hasil pertama Naruto	Lulus otomatis dan live; UI manual belum diuji
6	Hasil kosong	Fixture list kosong	Empty state tampil	Widget test menemukan “Anime tidak ditemukan”	Lulus otomatis
7	Navigasi detail	Klik kartu Naruto	Halaman detail terbuka	Detail Naruto terbuka dengan 220 episode dan sinopsis	Lulus runtime web
8	Detail dari API live	GET ID 5114	Informasi detail tampil	Tenrai live menghasilkan Fullmetal Alchemist: Brotherhood; mock endpoint juga lulus	Lulus live dan otomatis
9	Loading visual	Tahan request melalui browser automation	Progress indicator tampil	Indikator terlihat pada screenshot loading.png	Lulus uji terkontrol web
10	Error jaringan	Batalkan request API melalui browser automation	Pesan dan retry	Data gagal dimuat, pesan koneksi dan tombol Coba Lagi terlihat	Lulus uji terkontrol web
11	Retry	Tekan Coba Lagi	Request diulang	Belum diuji manual	Belum diuji
12	Refresh	Tekan refresh	Top Anime diminta ulang	Belum diuji manual	Belum diuji
13	Tombol kembali	Kembali dari detail	Home tampil	Belum diuji manual	Belum diuji
14	HTTP error	Status 429/500	Error terkontrol	Unit test kedua status lulus	Lulus otomatis
15	JSON rusak/null	Body invalid/tidak berisi data	Error parsing terkontrol	Unit test invalid, {} dan data null lulus	Lulus otomatis

No.	Fitur	Skenario	Hasil yang diharapkan	Hasil aktual	Status
16	Model nullable	Field opsional null	Nilai pengganti aman	Unit test lengkap dan null lulus	Lulus otomatis
17	Static analysis	`flutter analyze`	Tidak ada temuan	No issues found	Lulus

Endpoint publik `api.jikan.moe` sudah dihentikan, bukan kesalahan parsing aplikasi. Endpoint Tenrai pengganti telah diverifikasi live melalui service Flutter: Top Anime, pencarian Naruto, dan detail anime/5114 merespons HTTP 200 dengan data yang dapat diparsing. Pengujian visual aplikasi telah dilakukan pada build web lokal melalui Chrome dengan viewport mobile. Poster, daftar, pencarian, detail, loading, dan error telah dibuktikan melalui screenshot runtime. Pengujian Android fisik, refresh, retry setelah jaringan pulih, dan tombol kembali masih perlu dilengkapi bila dosen mensyaratkan perangkat Android. Detail langkah uji tersedia dalam `docs/PENGUJIAN.md`.

BAB III. PENUTUP

3.1 Kesimpulan

AniList Mini telah diimplementasikan sebagai aplikasi Flutter dengan halaman daftar/pencarian dan detail, model Dart, service HTTP, serta UI dark mode. Kode mengandung HTTP GET, pemeriksaan status, parsing JSON, Future, async/await, dan FutureBuilder untuk loading, error, dan data berhasil. Pengujian otomatis lokal mencakup model, service dengan respons uji, dan dua kondisi widget; seluruhnya lulus, `flutter analyze` tidak menemukan masalah, dan build web serta APK debug berhasil. Endpoint live Tenrai telah terkonfirmasi melalui pemeriksaan service, dan UI Home, pencarian, detail, loading serta error telah diperiksa melalui browser automation pada viewport 430 × 932 piksel. Uji Android fisik belum dilakukan. Tujuan penerapan teknis dibuktikan melalui kode, tes otomatis, request live, dan lima screenshot runtime. Pengujian browser tidak disamakan dengan pengujian perangkat Android fisik.

3.2 Saran

Pengembangan selanjutnya dapat menambah pagination untuk hasil panjang, favorit dengan penyimpanan lokal, filter genre, cache gambar/data, serta pengujian integrasi pada perangkat dan jaringan nyata. Fitur tersebut adalah usulan dan belum diimplementasikan pada versi ini.

DAFTAR PUSTAKA

1. Flutter. *Flutter documentation*. https://docs.flutter.dev/ (rujukan dokumentasi; tanggal akses belum diverifikasi).

2. Dart. *Dart documentation*. https://dart.dev/guides (rujukan dokumentasi; tanggal akses belum diverifikasi).

3. Dart packages. *http package*. https://pub.dev/packages/http (rujukan dokumentasi; tanggal akses belum diverifikasi).

4. Jikan. *Jikan API documentation*. https://docs.api.jikan.moe/ (acuan struktur API; tanggal akses perlu diisi saat finalisasi).

5. Tenrai. *Tenrai API documentation*. https://api.tenrai.org/documentation (pengganti kompatibel; tanggal akses perlu diisi saat finalisasi).

6. Flutter API. *FutureBuilder class*. https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html (rujukan dokumentasi; tanggal akses perlu diisi saat finalisasi).

Tanggal akses harus diisi setelah sumber benar-benar dibuka saat finalisasi.